

Final Project - Part 1: Language Design & Grammar

The Emerald Programming Language

1. Language Name & Description

Language Name: Emerald

Emerald is a programming language designed for the purpose of providing students with the knowledge needed to grasp the principles of interpreters and compilers, and it features characteristics such as static scoping, common programming features such as arithmetic, logical expressions, variables, conditionals (if/else), loops (while), functions with input parameters and output values, and block lexical scope.

2. List of Keywords

Vela reserves the following 12 keywords. Reserved words may not be used as identifiers.

Keyword	Purpose
let	Declares and initializes a variable
print	Outputs the value of an expression
if	Begins a conditional statement
else	Alternate branch of a conditional statement
while	Begins a pre-test loop
func	Defines a function
return	Returns a value from a function
true	Boolean literal
false	Boolean literal
and	Logical AND operator
or	Logical OR operator
not	Logical negation operator

3. Operators and Delimiters

3.1: Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction / unary negation
*	Multiplication

Operator	Meaning
/	Division

3.2: Comparison Operators

Operator	Meaning
==	Equal to
!=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

3.3: Logical Operators

Operator	Meaning
and	Logical AND
or	Logical OR
not	Logical NOT (unary)

3.4: Assignment Operator

= means assigns the value of an expression to a variable or array element.

Delimiters:

Symbol	Meaning
()	Grouping expressions; function parameter/argument lists; for-loop header
{ }	Block delimiters for if/else, while, for, and function bodies
[]	Array literal and array index access

Symbol	Meaning
,	Separates parameters, arguments, and array elements
;	Terminates a statement; separates the three clauses of a for-loop header

4. Variable Declaration & Assignment Syntax

Variables are declared with `let` and must be initialized at declaration. Re-assignment uses the identifier alone (no `let`).

```
let x = 10;    // declaration with initialization
x = x + 1;    // re-assignment
let name = 3.5; // numeric values may be integer or decimal
```

5. Control-Structure Syntax

if / else :

```
if (x < y) {
    print(x);
} else {
    print(y);
}
```

While loop:

```
while (x < 10) {
    x = x + 1;
}
```

for loop:

for uses the familiar C-style three-clause header: an initializer, a condition checked before each iteration, and an update executed after each iteration. The loop variable declared in the initializer is scoped to the loop only.

```
for (let i = 0; i < 5; i = i + 1) {
    print(i);
}
```

6. Function Syntax

Functions are declared with `func`, take zero or more comma-separated parameters, and return a value with `return`. Each function body introduces a new scope: parameters and any variables declared with `let` inside the body (including a for-loop's own init variable) are local to that call and are not visible outside it.

Variables not found in the local scope resolve to the enclosing (global) scope.

```
func add(a, b) {  
    let result = a + b; // result is local to add()  
    return result;  
}
```

```
let total = add(4, 5); // total == 9; 'result' is not visible here
```

7. SAMPLE PROGRAMS

Sample Program 1 — Arithmetic, Variables, and Print

```
let x = 10;  
let y = 20;  
let sum = (x + y) * 2;  
print(sum);          // prints 60
```

Sample Program 2 — Comparison and if/else

```
let a = 15;  
let b = 7;  
if (a > b) {  
    print(a);  
} else {  
    print(b);  
}          // prints 15
```

Sample Program 3: While Loop

```
let i = 0;  
let total = 0;  
while (i < 5) {  
    total = total + i;  
    i = i + 1;  
}  
print(total);      // prints 10
```

Sample Program 4: Function with Parameters, Return Value, and Scope

```
func square(n) {  
    let result = n * n;  
    return result;  
}
```

Initial BNF/EBNF grammar:

Notation: {P0} means zero-or-more repetitions of P0, [P0] means P0 is optional, and | separates alternatives. Terminal symbols (literal tokens) are shown in double quotes.

```
<program>      ::= { <statement> }  
  
<statement>    ::= <declaration>  
                | <assignment>  
                | <print-statement>  
                | <if-statement>  
                | <while-statement>  
                | <for-statement>  
                | <function-definition>  
                | <return-statement>  
                | <expression-statement>  
                | <block>  
  
<block>        ::= "{" { <statement> } "  
  
<declaration>  ::= "let" <identifier> "=" <expression> ","  
  
<assignment>   ::= <identifier> [ "[" <expression> "]" ] "=" <expression> ","  
  
<print-statement> ::= "print" "(" <expression> ")" ";"  
  
<if-statement>  ::= "if" "(" <expression> ")" <block> [ "else" <block> ]  
  
<while-statement> ::= "while" "(" <expression> ")" <block>  
  
<for-statement> ::= "for" "(" <for-init> ";" <expression> ";" <for-update> ")" <block>  
  
<for-init>     ::= "let" <identifier> "=" <expression>  
                | <identifier> "=" <expression>  
  
<for-update>   ::= <identifier> [ "[" <expression> "]" ] "=" <expression>  
  
<function-definition> ::= "func" <identifier> "(" [ <parameter-list> ] ")" <block>  
  
<parameter-list> ::= <identifier> { "," <identifier> }
```

```

<return-statement> ::= "return" [ <expression> ] ";"

<expression-statement> ::= <expression> ";"

<expression>      ::= <logical-or>

<logical-or>      ::= <logical-and> { "or" <logical-and> }

<logical-and>     ::= <equality> { "and" <equality> }

<equality>        ::= <comparison> { ( "==" | "!=" ) <comparison> }

<comparison>      ::= <arith-expr> { ( "<" | ">" | "<=" | ">=" ) <arith-expr> }

<arith-expr>      ::= <term> { ( "+" | "-" ) <term> }

<term>            ::= <factor> { ( "*" | "/" ) <factor> }

<factor>          ::= <unary>
                    | <function-call>
                    | <array-literal>
                    | <array-access>
                    | <number>
                    | <identifier>
                    | "true" | "false"
                    | "(" <expression> ")"

<unary>           ::= ( "-" | "not" ) <factor>

<function-call>   ::= <identifier> "(" [ <argument-list> ] ")"

<argument-list>   ::= <expression> { "," <expression> }

<array-literal>   ::= "[" [ <expression> { "," <expression> } ] "]"

<array-access>    ::= <identifier> "[" <expression> "]"

<number>          ::= <digit> { <digit> } [ "." <digit> { <digit> } ]

<identifier>      ::= <letter> { <letter> | <digit> | "_" }

<letter>          ::= "a" | ... | "z" | "A" | ... | "Z"

<digit>           ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

```

The order of operator precedence from least to most significant is: or, and, equality (==, !=), comparison (<, >, <=, >=), addition (+, -), multiplication (*, /), and negation (-, not). The presence of parentheses takes priority over standard operator precedence. Moreover, the process of array indexing or the invocation of functions will take precedence over the other operations. The loop variable of the for-loop has a limited scope within the loop, just like the variable's scope in a function.

Proposed Additional / Extension Feature

Extension feature: 1-dimensional arrays (such as lists). There are several reasons for choosing arrays as an extension feature. First, they allow related values (like scores) to be grouped together and traversed using the already existing while/for loops, thus minimizing the need for creating new control flow mechanisms. We declare an array using brackets (let scores = [90, 85, 100];), access the values in it using indexing (scores[0]), and set new values using indexed assignment (scores[1] = 100;). As a result, the grammar system includes an indexing alternative for , , and . Nevertheless, the for loop is considered as a primary control mechanism along with if/else and while loops and does not fall into the scope of the current group's extension feature; arrays, therefore, remain the only extension feature used for the grading purposes. The implementation process will involve zero-based indexing, as well as bounds checking during execution and error reporting. We may consider including an additional length(array) function if time allows.

Initial Group Member Responsibilities:

Member	Responsibilities
Shohag	Initialize the programming language name and list of operators, keywords. Cross-check with the other member after complete the BNF/EBNF rules
Prashant	Write the BNF and EBNF rules
Abir	Write the BNF and EBNF rules
Toufique	Write reports; cross-check BNF/EBNF rules after writing the rules
Aman	GitHub repository setup, suggest adding a for loop in the BNF/EBNF rules, and cross-check the full assignment after writing the complete report

AI usage:

We use the AI tools for suggestions. During the BNF/EBNF, we got stuck on some points, so we asked ChatGPT what the wrong rule was.